MODULE *Overwatch*

Section: Imports

Shared operators used by this module are imported below.

EXTENDS *Naturals*, *FiniteSets*, *Sequences*, *FailurePolicy*

Section: State and result domains

Workspace versions, completed outcomes, and message records form the result layer. Finite bounds are supplied by the *TLC* configuration, not by the runtime.

CONSTANTS *Names*, *MaxEdits*, *Timeout*

ASSUME "timeout" $\notin$ *Names*

$Versions \triangleq 0 \,..\, MaxEdits$

$Results \triangleq \text{SUBSET } Names$

$TimedOut \triangleq$ "timeout"

$Outcomes \triangleq \text{SUBSET } (Names \cup \{TimedOut\})$

$NoMessage \triangleq [held \mapsto \text{FALSE},\ waking \mapsto \text{FALSE},\ failures \mapsto \{\}]$

$NotTold \triangleq [kind \mapsto \text{"none"},\ fresh \mapsto \{\}]$

Section: State shape

The checked version, unread results, inherited baseline, and delivery state are separate. *TypeOK* constrains their domains without advancing an execution.

VARIABLES
truth, *head*, *based*, *busy*, *from*, *quiet*, *installed*,
produced, *consumed*, *pending*, *report*, *baseline*, *wasFailing*,
outbox, *told*, *flush*, *turn*, *toldThisTurn*, *promptQueued*, *knows*, *elapsed*, *edited*

$vars \triangleq \langle truth, head, based, busy, from, quiet, installed,$
$produced, consumed, pending, report, baseline, wasFailing,$
$outbox, told, flush, turn, toldThisTurn, promptQueued, knows, elapsed, edited\rangle$

$TypeOK \triangleq$
$\land\ truth \in [Versions \to Results]$
$\land\ head \in Versions \land based \in 0 \,..\, head$
$\land\ busy \in \text{BOOLEAN} \land from \in Versions \land quiet \in \text{BOOLEAN} \land installed \in \text{BOOLEAN}$
$\land\ produced \in 0 \,..\, (MaxEdits + 1) \land consumed \in 0 \,..\, produced$
$\land\ pending \in Seq([id : 1 \,..\, (MaxEdits + 1),\ at : Versions,\ failures : Outcomes])$
$\land\ Len(pending) \leq MaxEdits + 1$
$\land\ report \in [known : \text{BOOLEAN},\ id : 0 \,..\, produced,\ at : Versions,\ failures : Outcomes]$
$\land\ baseline \in [known : \text{BOOLEAN},\ failures : Outcomes]$
$\land\ wasFailing \in \text{BOOLEAN}$
$\land\ outbox \in [held : \text{BOOLEAN},\ waking : \text{BOOLEAN},\ failures : Outcomes]$
$\land\ told \in [kind : \{\text{"none"}, \text{"failed"}, \text{"recovered"}\},\ fresh : Outcomes]$
$\land\ flush \in \text{BOOLEAN}$
$\land\ turn \in \{\text{"idle"}, \text{"running"}\}$
$\land\ toldThisTurn \in \text{BOOLEAN} \land promptQueued \in \text{BOOLEAN}$
$\land\ knows \in Outcomes \land elapsed \in 0 \,..\, Timeout$
$\land\ edited \in \text{BOOLEAN}$

Section: Failure gating

A failure report becomes a gate only when it is a regression against the session baseline, or when the check times out.

$Failing \triangleq report.known \land report.failures \neq \{\}$

$Regressed(f) \triangleq TimedOut \in f \lor (baseline.known \land NewFailure(f,\ baseline.failures))$

$Guard \triangleq Failing \land Regressed(report.failures)$

Section: Initial state

The first check starts at the inherited workspace version. No result or baseline is known and no notification has been sent.

```
Init ≜
    ∧ truth ∈ [Versions → Results]
    ∧ head = 0 ∧ based = 0
    ∧ busy = TRUE ∧ from = 0 ∧ quiet = FALSE ∧ installed = FALSE
    ∧ produced = 0 ∧ consumed = 0 ∧ pending = ⟨⟩
    ∧ report = [known ↦ FALSE, id ↦ 0, at ↦ 0, failures ↦ {}]
    ∧ baseline = [known ↦ FALSE, failures ↦ {}]
    ∧ wasFailing = FALSE
    ∧ outbox = NoMessage ∧ told = NotTold ∧ flush = FALSE
    ∧ turn = "idle" ∧ toldThisTurn = FALSE ∧ promptQueued = FALSE
    ∧ knows = {} ∧ elapsed = 0 ∧ edited = FALSE
```

Section: Result delivery and baseline isolation

Queue deduplicates notification signatures. Deliver consumes a completed snapshot, establishes the inherited baseline, and queues a regression or recovery. A recovery can cancel a failure message that has not yet been delivered.

```
Queue(signature, message) ≜
    IF told = signature ∧ ¬outbox.held
     THEN UNCHANGED ⟨outbox, told, flush⟩
     ELSE  ∧ told′ = signature ∧ outbox′ = message ∧ flush′ = TRUE

Deliver(v, id, f) ≜
    LET b ≜ IF baseline.known THEN baseline.failures ELSE IF head = based THEN f ELSE {}
        isRegression ≜ TimedOut ∈ f ∨ ¬(f ⊆ b)
    IN  ∧ report′ = [known ↦ TRUE, id ↦ id, at ↦ v, failures ↦ f]
        ∧ baseline′ = IF TimedOut ∈ f THEN baseline ELSE [known ↦ TRUE, failures ↦ b]
        ∧ wasFailing′ = isRegression
        ∧ IF isRegression
            THEN Queue([kind ↦ "failed", fresh ↦ f \ b],
                       [held ↦ TRUE, waking ↦ edited ∨ TimedOut ∈ f, failures ↦ f])
          ELSE IF ¬wasFailing
            THEN UNCHANGED ⟨outbox, told, flush⟩
          ELSE IF outbox.held
            THEN ∧ outbox′ = NoMessage ∧ told′ = NotTold ∧ UNCHANGED flush
          ELSE Queue([kind ↦ "recovered", fresh ↦ {}], [held ↦ TRUE, waking ↦ FALSE, failures ↦ f])
```

Section: Workspace changes and scheduling

An agent edit advances head without moving based, and marks the next read as following the agent's own change. Another process can change the workspace at any time without committing; *OutsideEdit* advances head the same way but marks nothing. An external head change moves both and invalidates the baseline and unread results. *QuietElapses* starts the next check; *Tick* advances its timeout clock.

```
Edit ≜
    ∧ turn = "running" ∧ head < MaxEdits
    ∧ head′ = head + 1 ∧ quiet′ = TRUE ∧ edited′ = TRUE
    ∧ UNCHANGED ⟨truth, based, busy, from, installed, produced, consumed, pending,
                 report, baseline, wasFailing, outbox, told, flush, turn,
                 toldThisTurn, promptQueued, knows, elapsed⟩

OutsideEdit ≜
    ∧ head < MaxEdits
    ∧ head′ = head + 1 ∧ quiet′ = TRUE
    ∧ UNCHANGED ⟨truth, based, busy, from, installed, produced, consumed, pending,
                 report, baseline, wasFailing, outbox, told, flush, turn,
                 toldThisTurn, promptQueued, knows, elapsed, edited⟩

HeadMoves ≜
    ∧ head < MaxEdits
    ∧ head′ = head + 1 ∧ based′ = head + 1
```

```
    ∧ baseline′ = [known ↦ FALSE, failures ↦ {}]
    ∧ quiet′ = TRUE ∧ pending′ = ⟨⟩
    ∧ from′ = IF busy THEN head + 1 ELSE from
    ∧ elapsed′ = IF busy THEN 0 ELSE elapsed
    ∧ UNCHANGED ⟨truth, busy, installed, produced, consumed, report, wasFailing, outbox,
                 told, flush, turn, toldThisTurn, promptQueued, knows, edited⟩

QuietElapses ≜
    ∧ quiet ∧ installed
    ∧ quiet′ = FALSE ∧ busy′ = TRUE ∧ from′ = head ∧ elapsed′ = 0
    ∧ UNCHANGED ⟨truth, head, based, installed, produced, consumed, pending,
                 report, baseline, wasFailing, outbox, told, flush, turn,
                 toldThisTurn, promptQueued, knows, edited⟩

Tick ≜
    ∧ busy ∧ elapsed < Timeout ∧ elapsed′ = elapsed + 1
    ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced,
                 consumed, pending, report, baseline, wasFailing, outbox,
                 told, flush, turn, toldThisTurn, promptQueued, knows, edited⟩
```

Completion appends a result; it does not deliver it. Partial output is a stuttering step at this abstraction: only a complete build enters pending.

Section: Buffered completion and consumption

Run completion appends a snapshot to pending without publishing it. *ReadResults* consumes the newest completed snapshot and clears the buffer. A later incomplete build does not replace an earlier complete result.

```
Buffer(v, f) ≜
    ∧ produced′ = produced + 1
    ∧ pending′ = Append(pending, [id ↦ produced + 1, at ↦ v, failures ↦ f])
    ∧ busy′ = FALSE ∧ installed′ = TRUE
    ∧ UNCHANGED ⟨truth, head, based, from, quiet, consumed, report, baseline,
                 wasFailing, outbox, told, flush, turn, toldThisTurn, promptQueued, knows, elapsed, edited⟩

RunEnds ≜ busy ∧ ∃ v ∈ from .. head : Buffer(v, truth[v])
TimesOut ≜ busy ∧ elapsed = Timeout ∧ Buffer(from, {TimedOut})

ReadResults ≜
    ∧ pending ≠ ⟨⟩
    ∧ LET latest ≜ pending[Len(pending)]
      IN  Deliver(latest.at, latest.id, latest.failures)
    ∧ consumed′ = produced ∧ pending′ = ⟨⟩ ∧ edited′ = FALSE
    ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced,
                 turn, toldThisTurn, promptQueued, knows, elapsed⟩
```

Section: Notification delivery and turn completion

A running turn receives at most one notification before another turn starts. An idle agent can receive one queued wakeup. Flush publishes eligible messages; *TurnCompletes* keeps a guarded turn running and permits a clean turn to settle.

```
Held ≜ IF turn = "running" THEN toldThisTurn ELSE promptQueued ∨ ¬outbox.waking

Flush ≜
    ∧ flush ∧ flush′ = FALSE
    ∧ IF outbox.held ∧ ¬Held
        THEN ∧ outbox′ = NoMessage ∧ knows′ = outbox.failures
             ∧ IF turn = "running"
                 THEN toldThisTurn′ = TRUE ∧ UNCHANGED promptQueued
                 ELSE promptQueued′ = TRUE ∧ UNCHANGED toldThisTurn
        ELSE UNCHANGED ⟨outbox, knows, toldThisTurn, promptQueued⟩
    ∧ UNCHANGED ⟨truth, head, based, busy, from, quiet, installed, produced, consumed,
                 pending, report, baseline, wasFailing, told, turn, elapsed, edited⟩
```

$$TurnStarts \triangleq$$
$$\quad \land\ turn = \text{“idle”} \land turn' = \text{“running”}$$
$$\quad \land\ toldThisTurn' = promptQueued \land promptQueued' = \text{FALSE} \land flush' = \text{TRUE}$$
$$\quad \land\ \text{UNCHANGED}\ \langle truth,\ head,\ based,\ busy,\ from,\ quiet,\ installed,\ produced,\ consumed,$$
$$\qquad pending,\ report,\ baseline,\ wasFailing,\ outbox,\ told,\ knows,\ elapsed,\ edited\rangle$$

$$WokenTurnStarts \triangleq promptQueued \land TurnStarts$$

$$TurnCompletes \triangleq$$
$$\quad \land\ turn = \text{“running”}$$
$$\quad \land\ \text{IF}\ Guard$$
$$\qquad \text{THEN}\ \land\ toldThisTurn' = \text{TRUE} \land knows' = report.failures \land \text{UNCHANGED}\ \langle turn,\ flush\rangle$$
$$\qquad \text{ELSE}\ \land\ turn' = \text{“idle”} \land flush' = \text{TRUE} \land \text{UNCHANGED}\ \langle toldThisTurn,\ knows\rangle$$
$$\quad \land\ \text{UNCHANGED}\ \langle truth,\ head,\ based,\ busy,\ from,\ quiet,\ installed,\ produced,\ consumed,$$
$$\qquad pending,\ report,\ baseline,\ wasFailing,\ outbox,\ told,\ promptQueued,\ elapsed,\ edited\rangle$$

Section: Allowed executions and fairness

Next chooses an enabled transition. Spec permits stuttering and requires weak fairness for the listed progress actions; this does not impose deadlines.

$$Next \triangleq Edit \lor OutsideEdit \lor HeadMoves \lor QuietElapses \lor Tick \lor RunEnds \lor TimesOut$$
$$\qquad \lor ReadResults \lor Flush \lor TurnStarts \lor TurnCompletes$$

$$Spec \triangleq$$
$$\quad \land\ Init \land \Box[Next]_{vars}$$
$$\quad \land\ \mathrm{WF}_{vars}(QuietElapses) \land \mathrm{WF}_{vars}(Tick) \land \mathrm{WF}_{vars}(TimesOut)$$
$$\quad \land\ \mathrm{WF}_{vars}(ReadResults) \land \mathrm{WF}_{vars}(Flush)$$
$$\quad \land\ \mathrm{WF}_{vars}(WokenTurnStarts) \land \mathrm{WF}_{vars}(TurnCompletes)$$

Section: Safety and progress properties

Results retain their ordering and checked version. The baseline cannot absorb later edits. A failure wakes the agent only when its read follows the agent's own edit, or when the check timed out, so another process's changes never wake it. Under *Spec* fairness, checks end, and each regression reaches the agent or waits, without waking it, for its next turn. These properties are model-checked within the committed configuration bounds.

$$BufferedResultsAreOrdered \triangleq$$
$$\quad \land \forall\, i \in 1\,..\,Len(pending) : pending[i].id > consumed \land pending[i].id \leq produced$$
$$\quad \land \forall\, i \in 1\,..\,(Len(pending) - 1) : pending[i].id < pending[i+1].id$$

$$ReadsSelectLatestComplete \triangleq report.known \Rightarrow report.id = consumed$$
$$ResultsMatchBuilds \triangleq report.known \Rightarrow report.failures = truth[report.at] \lor report.failures = \{TimedOut\}$$
$$BaselineNeverAbsorbsEdits \triangleq baseline.known \Rightarrow baseline.failures \subseteq truth[based]$$
$$RunsEnd \triangleq busy \leadsto \neg busy$$
$$EveryEditIsChecked \triangleq \Diamond\Box(report.known \land report.at = head)$$
$$Regressions \triangleq truth[head] \setminus truth[based]$$
$$WaitsForTurn \triangleq turn = \text{“idle”} \land outbox.held \land \neg outbox.waking \land Regressions \subseteq outbox.failures$$
$$RegressionsReachAgent \triangleq \Diamond\Box(Regressions \subseteq knows \lor TimedOut \in knows \lor WaitsForTurn)$$
$$OutsideChangesNeverWake \triangleq$$
$$\quad \Box[told' \neq told \land told'.kind = \text{“failed”} \land outbox'.waking$$
$$\qquad \Rightarrow edited \lor TimedOut \in outbox'.failures]_{vars}$$